

דו"ח מחקר

ממ"ן 06

מגישים: תומר טולדו 209706795 ואביאל כהן 205735806

מבוא לאבטחת המרחב המקוון – סמסטר 2026א

האוניברסיטה הפתוחה

תוכן עניינים

3	תקציר ומוטיבציה
3	רקע
5	תצורה ניסויית
6	מתודולוגיה
8	תוצאות
11	ניתוח ודיון
14	שיקולים אתיים
15	מסקנות ועבודה עתידית
16	מקורות
נספחים (לוגים, טבלאות ותרשימים) – כלל הלוגים נמצאים בעמוד ה- GitHub של הפרויקט.	

תקציר ומוטיבציה

דוח זה מסכם את הפרויקט בקורס מבוא לאבטחת המרחב המקוון, שמטרתו לבצע ניסוי הניתן לשחזור, המשווה בין מנגנוני גיבוב ואימות סיסמאות (SHA256, BCrypt, Argon2ID) ובוחן את השפעת מנגנוני הגנה נפוצים במערכות הזדהות (Rate Limiting, Pepper, CAPTCHA, Lockout) וכן דיון בהקשר של אימות רב שלבי (TOTP).

הבחירה במנגנונים אלו משקפת מציאות - מנגנון הגיבוב מגן בעיקר מפני מתקפות offline לאחר דליפת בסיס נתונים (מכיוון שבעזרתן לא שומרים סיסמה raw בתוך ה-database), בעוד שמנגנוני ההגנה "סביב" האימות קובעים את קצב ואפקטיביות מתקפות online כגון Brute-Force, Password Spray, וחלק מהרכיבים גם עבור DDoS (לא נבדק בפרויקט זה).

מבחינה מקצועית, הפרויקט נועד להמחיש את ה-Tradeoff בין אבטחה, שימושיות וביצועים: האצה / האטה של אימות סיסמה משפיעה על שרידות מול ניסיונות ניחוש, אך גם על חוויית המשתמש ועל צריכת משאבים. בהמשך הדוח מוצגים: תצורת הניסוי, המתודולוגיה, תוצאות כמותיות (זמני תקיפה, קצב ניסיונות, אחוזונים), ניתוח תוצאות והערכת תוקף הניסוי, שיקולים אתיים, והצעות להמשך עבודה.

רקע

גיבוב סיסמאות – "מהיר" מול "איטי" / Memory Hard

פונקציות גיבוב כלליות ומהירות (כמו SHA256) אינן מיועדות לאחסון סיסמאות, משום שהן מאפשרות קצב ניחוש גבוהה מאוד (בעיקר במתקפות offline, אך גם ב-online במערכת ללא הגנות סביב האימות) כגון Rate Limit, TOTP, וכו'.

לעומת זאת, פונקציות ייעודיות לסיסמאות (כמו BCrypt ו-Argon2ID) מכוונות להיות "יקרות" חישובית (ובפרט אצל Argon2ID גם עטירות זיכרון), כדי להקשות על ניסיונות ניחוש מסיביים.

¹OWASP ממליץ להשתמש באלגוריתמים מודרניים כגון Argon2id או BCrypt ומסביר את התפקיד של salt ושל pepper בתכנון אחסון סיסמאות.

Argon2 (ובפרט Argon2ID), הוגדר כ-memory hard במטרה להקשות על שימוש בחומרה ייעודית / מקבילית בניחוש סיסמאות, ותקן RFC 9106² מסכם את עקרונותיו ואת פרמטרי התצורה הנפוצים.

BCrypt הוצע כבר בסוף שנות ה-90 כגישה "Future Adaptable" המאפשרת התאמת עלות לאורך זמן באמצעות cost factor, כדי להישאר רלוונטי גם כאשר החומרת משתפרת.

¹[OWASP Password Storage Cheat Sheet](#) – OWASP, ראשי תיבות של Open Worldwide Application Security Project היא קהילה מקוונת ללא מטרות רווח, המפרסמת מידע ומשאבים בקוד פתוח בנושאי IoT ואבטחת תוכנה ויישומי Web.
²[RFC 9106](#) - Argon2 Memory-Hard Function for Password Hashing and Proof-of-Work Applications

Pepper ו-Salt

Salt פר-משתמש מונע זיהוי סיסמאות זהות לפי hash ומקשה על התקפות בעזרת Rainbow Tables³. ב-BCrypt / Argon2ID, ה-salt משולב בתכנון ומיושם כחלק מהאלגוריתם באופן סטנדרטי.

Pepper הוא רכיב סודי נוסף שמוחזק מחוץ לבסיס הנתונים (למשל בקונפיגורציה / HSM). ערכו המרכזי הוא בהפחתת נזק במקרה של דליפת DB בלבד; גם אם התוקף מחזיק את ה-hashes, ללא ה-pepper קשה לו לאמת / לחשב ניחשים באופן מלא. OWASP מציין את pepper כשכבת אבטחה נוספת (defense-in-depth).

מודל תקיפה: Password Spraying ו-Brute Force

Brute-Force – ניסיונות שיטתיים לניחוש סיסמה עבור חשבון ספציפי.

Password Spraying – ניסיון של סיסמה אחת / רשימה קטנה על פני הרבה שמות משתמשים כדי להימנע מנעילת חשבון ממוקדת.

הגנות סביב האימות

Rate Limiting / Throttling – הגבלת מספר ניסיונות כושלים בחלון זמן כדי להפוך ניחוש בקנה מידה גדול לבלתי מעשי.

Lockout – נעילת חשבון לאחר מספר כשלונות רצופים. OWASP מזהיר⁴ שמנגנון נעילה עלול לשמש לתקיפת DoS על משתמשים (שליחה של אלפי בקשות למשתמשים לגיטימיים ונעילתם).

CAPTCHA – אתגר שמטרתו להבחין בין אדם לאוטומציה ולהעלות עלות לתוקף. OWASP מציג CAPTCHA כאחת ההגנות⁵ האפשריות כחלק ממערכת הגנות נגד מתקפות Brute Force.

TOTP – סיסמה חד פעמית מבוססת זמן, תקן RFC 6238⁶, המספק שכבת MFA שמונעת התחברות גם אם סיסמה נחשפה.

³ Rainbow Tables – מקפת Rainbow Tables הינה מתקפה המשתמשת בטבלאות מוכנות מראש על מנת לפרוץ גיבובי סיסמאות ב-Databases.

⁴ Authentication Cheat Sheet - Guidelines מרכזיים עבור תהליך ה-User Authentication
⁵ Blocking Brute Force Attacks - Guidelines מרכזיים ונקודות כשל עיקריות בחסימת Brute Force Attacks.

⁶ RFC 6238 - TOTP: Time-Based One-Time Password Algorithm

תצורה ניסויית

יישויות וקבצים

בעבור כל Test נשמרו תוצרי ההרצה הבאים:

1. Config.Json – מצב ההגנות הפעיל בDatabase (אלגוריתם גיבוב, Captcha, Lockout, Rate Limit, Pepper).
2. Brute_Force.jsonl / Spray_log.jsonl – לוג התקיפה בפועל (ניסיון מספר N , שם משתמש, סטטוס HTTP, Latency, דגלי הגנות פעילים וכו').
3. Resources.jsonl – דגימות שימוש משאבים (Threads, זיכרון, CPU).
4. Attempts.jsonl – כלל הבקשות שהתקבלו בצד השרת (לא שימש בבסיס עיקרי לניתוח בדו"ח זה).

מגבלות עצירה

בברירת המחדל: לכל תצורה עד 50,000 ניסיונות או עד שעתיים ריצה (המוקדם מביניהם).

פרמטרים (ברירת מחדל לפי הדרישות)

- SHA256 + Per-User salt
- BCrypt: $Cost = 12$
- Argon2ID: $Time = 1, Memory = 64MB, Parallelism = 1$
- Rate Limit: $WindowSeconds = 60, MaxAttemptsPerUser = 5$
- Lockout: $LockoutThreshold = 10, Duration = 50,000 (\infty)$
- Captcha: $CaptchaAfterFailedAttempts = 3$

הפרמטרים נבחרו בבסיס השוואתי, בעוד ש-RFC 9106⁷ מספק הקשר והמלצות כלליות לצורת Argon2ID.

סט המשתמשים עבור בחינה

עבור הפרויקט שלנו, נבחר סט של 30 משתמשים, המחולקים ל-3 קטגוריות – התלויות ב'עוצמת' ססמותיהם.

Weak_*	סיסמאות המורכבות ממספרים או אותיות קטנות בלבד
Medium_*	סיסמאות המורכבות ממספרים, אותיות קטנות או גדולות בלבד
Strong_*	סיסמאות המורכבות ממספרים, אותיות קטנות או גדולות ותווים מיוחדים.

⁷RFC 9106 - Argon2 Memory-Hard Function for Password Hashing and Proof-of-Work Applications

תצורות שנבחנו בדוח זה (הסט העיקרי)

הניתוח מתמקד בתתי-הבדיקות הבאות:

1. Brute-Force ללא הגנות:

Test01	SHA256
Test03	BCrypt
Test05	Argon2ID

2. Brute-Force עם Pepper בלבד:

Test02	SHA256 + Pepper
Test04	BCrypt + Pepper
Test06	Argon2ID + Pepper

3. Brute-Force עם הגנות נוספות פעילות (בדיקות השפעה יחידתיות):

Test10	SHA256 + Rate Limit
Test13	SHA256 + Lockout
Test15	SHA256 + CAPTCHA

4. Password-Spray על Argon2ID:

Test20	Argon2ID + Pepper
Test23	Argon2ID + Pepper + Rate Limit + Lockout + CAPTCHA

חשוב! ברירת המחדל עבור תקיפת Password Spray בפרויקט שלנו בוצעה בעזרת 10 תהליכונים (Threads).

מתודולוגיה

איסוף נתונים ושדות נמדדים

מכל שורת jsonl בלוג התקיפה נשלפו:

- AttemptNumber, HTTPStatus, Attempt.Success, Attempt.HashAlgorithm
- דגלי הגנות (CaptchaRequired, TOTPRequired) וסטטוסי הגנה (למשל 423/429).

מ-resources.jsonl (בעיקר ב-spray) נמדדו:

- CPUPercent
- WorkingSetBytes

מדדים סטטיים

לכל ריצה חושבו:

- זמן ריצה כולל – $\max(timestamp) - \min(timestamp)$
- קצב ניסיונות ($attempt/sec$) – מופיע בסוף כל קובץ תקיפה ממוצע הזמן עבור כל בקשה.
- Latency – ממוצע וחציון.
- תוצאה – הצלחה (200), כשל (401), Locked (423), Rate Limited (429)

לאחר מכן הוצגו סיכומים "ברמת תצורה" באמצעות חציון של מדדי הריצות, כדי לצמצם השפעת חריגים ולהציג התנהגות טיפוסית.

השוואת התפלגויות

בדיון ובפרשנות הושם דגש על:

- שינוי בסדרי גודל של קצב ניסיונות בין אלגוריתמים (ללא הגנות).
- מעבר מהתנהלות ללא הגבלות קצב (ללא Rate Limit / CAPTCHA / Lockout), לעבוד עם הגבלות, שבו הזמן עד הצלחה מתארך משמעותית.
- ב-Spray ; שינוי Coverage – כמה זוגות username / password מצליח התוקף לבדוק בפועל.

שחזור הניסוי

לשחזור מלא נדרשים:

1. שימוש בתוכנות IronGate⁸ - פרויקט קוד ייעודי המכיל:
 - א. IronGate.Core – מסד נתונים SQL המוקם בסביבה וירטואלית.
 - ב. IronGate.API – המאפשר התקשרות עם הסביבה הוירטואלית של המסד נתונים.
 - ג. IronGate.CLI – תוכנת תקיפה ייעודית המאפשרת בדיקות Brute-Force ו-Password Spray עם ה-API הנ"ל.
2. שימוש בקבצי הקונפיגורציה הייחודיים עבור כל טסט.
3. רשימת משתמשים וסיסמאות (קבצי seed) זהים.
4. שימוש עקבי ב-GROUP_SEED ופרמטרים זהים בכל הרצה.

תוצאות

להלן סיכום תוצאות עיקריות (המספרים מחושבים מתוך הלוגים המסופקים ב-Github⁹ הפרויקט)

Brute-Force ללא הגנות – השוואת אלגוריתמים

Test	Average Latency (ms)	Attempts / Sec
Test01 (SHA256)	12.0	31.47
Test03 (BCrypt)	290.75	3.32
Test05 (Argon2ID)	135.5	6.63

ממצא מרכזי – מעבר מ-SHA256 ל-BCrypt / Argon2ID מוריד את קצב הניסיונות בסדר גודל משמעותי.

Brute-Force עם Pepper בלבד – השוואת אלגוריתמים

Test	Average Latency (ms)	Attempts / Sec
Test02 (SHA256)	10.5	35.52
Test04 (BCrypt)	293.0	3.29
Test06 (Argon2ID)	132.0	6.99

הערה פרשנית (מפורטת בדיון) – התוצאה אינה מוכיחה ש-Pepper "מאיץ"; סביר שהשונויות בין ריצות / עומסים גדולה מהשפעת Pepper (שאמורה להיות זניחה יחסית לעלות). OWASP מציג¹⁰ את Pepper כשכבת אבטחה נגד דליפות DB ולא כמנגנון Throttling.

⁸ IronGate - פרויקט המדמה את סביבת מסד הנתונים, והגנות, ותוכנה ייעודית המאפשרת תקיפה דרך פעולות CLI. הסביבה הינה וירטואלית המכילה מסד נתונים SQL. כלל הפרויקט נכתב בשפת C# ופותח בעזרת כלים של .NET ו-Entiy Framework Core.
⁹ IronGate/docs/Test Logs - קבצי הלוג של הטסטים הנבחרים.
¹⁰ Pepper - הדרך בה OWASP מגדירים את השימוש ב-Pepper.

Test10 – Rate Limiting עם Brute-Force

- 5 ניסיונות לכל 60 שניות
- $RateLimitWindowSeconds = 60, MaxAttemptPerUser = 5$
- נצפו סטטוסים 401 לצד 429 (Rate Limited), ובמקרים מסוימים הצלחה (200) לאחר זמן ממושך.
- דוגמה מובהקת – עבור המשתמש weak_06, הצלחה הופיעה בניסיון 614, לאחר כ-6183 שניות (43:~1 שעות) עם 102 תגובות 429 בדרך (102 השהיות).

ממצא מרכזי – גם כאשר SHA256 מאפשר אימות מהיר, Rate Limiting הופך התקפות Brute-Force ארוכים לבלתי יעילים בזמן ריצה נתון.

Test 13 – Lockout עם Brute-Force

- סף נעילה $LockoutThreshold = 10$, ומשך הנעילה אינסופי (לפי הגדרות הפרויקט), לכן בפועל הכלי עוצר על נעילה.
- ברוב הריצות התקיפה נעצרה סביב 11-12 ניסיונות עם $HTTPStatus 423$ (נעילה ללא הצלחה).
- נרשמו 2 הצלחות מתוך 10 משתמשים, כאשר הסיסמה הנכונה הופיעה מוקדם מאוד (ניסיון 4 או 5).

ממצא מרכזי – Lockout הוא מנגנון יעיל מאוד נגד התקפות Brute-Force ממוקדות, אך דורש דיון זהיר לגבי שימושויות ו-DoS.

Test 15 – CAPTCHA עם Brute-Force

- $CaptchaFailedAfterAttempts = 3, CaptchaEnabled = True$
- במסגרת הפרויקט הוגדר שימוש אחר במנגנון ה-CAPTCHA. כאשר נדרשת CAPTCHA, הושג אחד תקין מנקודת הקצה $GET admin/captcha?group_seed$. בפועל, מנגנון ה-CAPTCHA ימנע כניסות למשתמשים בעקבות שקשה מאוד לשחזר / ליצור אחד תואם לשרת.
- לפי הנ"ל, ניתן לקטלג את כלל הכניסות בהם $CaptchaRequired = True$ ל-כניסות False Positive, מכיוון שבמנגנון הגנה אמיתי, לא יהיה גישה לקוד CAPTCHA תקין.
- ניכר כי הגנה זו האטה את קצב הניסיונות לאורך זמן, בעיקר בולט בריצה הריצה למשך זמן (משתמש weak_01).

ממצא מרכזי - מימוש CAPTCHA במערכת הניסוי חסם בהצלחה כלי תקיפה אוטומטי, אך נדרש דיון על עקיפה אפשרית / פגיעה בחוויית משתמש.

Argon2ID על Password Spraying

Test23 – Argon2ID + Pepper

- 23,171 ניסיונות במשך שעותיים.
- 4 הצלחות, כאשר ההצלחה הראשונה הייתה בניסיון 48, לאחר כ-28.4 שניות.

Test 20 – Fully Stacked (Argon2ID + Pepper + Rate Limit + Lockout + CAPTCHA)

- 128 ניסיונות במשך כ-209.6 שניות.
- 2 הצלחות, הצלחה ראשונה בניסיון 47 לאחר 11.7 ~ שניות.
- סטטוסים 401 (97 מופעים), 423 (15), 429 (13), 200 (2)
- CAPTCHA Required ב-80 ניסיונות.

משאבים (מתוך resources.jsonl)

- Test23 – CPU ממוצע 49% , ~1122MB Working Set
- Test20 – CPU ממוצע 4.47% , ~1077MB Working Set

ממצא מרכזי – בהתקפת Password Spray, ההבדל הדרמטי אינו רק "כמה מהיר ה-Hash", אלא בעיקר כמה ניסיונות מותר לתוקף לבצע לפני שהוא מקבל סטטוס 429/423/CAPTCHA. זאת בדיוק ההתנהגות המאפיינת את סוג תקיפה זו, ניסיון להימנע מנעילות ממוקדות, ולכן הגנה משולבת משנה את כללי המשחק.

ניתוח ודיון

תפקיד האלגוריתם מול תפקיד ההגנות סביב האימות

כאשר אין הגנות סביב האימות, האלגוריתם הוא הגורם הדומיננטי בקצב הניחוש:

- כצפוי, SHA256 מאפשר קצב גבוה מאוד בריצה רציפה.
- Argon2ID ו-BCrypt מורידים את הקצב באופן משמעותי בריצה רציפה, ולכן מגדילים את ה"זמן כיסוי" של מרחב הסיסמאות.

כאשר מוסיפים הגנות סביב האימות, הן אמורות "לשבור" את הקשר הישיר בין עלות ה-Hash לבין קצב התקיפה (למשל Rate Limiting / Lockout יכולים להוריד קצב גם אם hash מהיר).

קשר זה אכן נשבר בטסטים 10,13,15 (אל מול טסט 1), כאשר הוספנו כל פעם הגנה שונה, ובפרט בהגנת Full Stack בטסט 20. ניתן לקחת כדוגמה את המשתמש `weak_06`, אשר קצב הפריצה משתנה בצורה דרסטית בין הטסטים השונים.

Test	Total Amount of Requests	Attempts / Sec	Time to First Success
Test01	512	32~	16~ sec
Test10	614	6.26~	1:38~ hr
Test13	11	11	Locked Out
Test15	512	16~	32~ sec

Pepper – איך לפרש את התוצאות

מבחינה עקרונית, Pepper נועד לשפר עמידות במקרה של דליפת DB, ולא להיות מנגנון Throttling של מתקפות Online.

בתוצאות שלנו ניתן להבחין בכך, מכיוון שבכל מצבי ההגנה, ה-Latency לא משתנה משמעותית בהשוואה לריצות ללא Pepper. עם זאת, בריצות רציפות וארוכות, ניתן לראות שינוי, אך שינוי זה עלול להיות גם כתוצאה ממשאבי המערכת הדרושים באותה עת.

Rate Limiting מול Lockout – יעילות מול שימושיות

Rate Limiting (5 ניסיונות לדקה) הגדיל דרמטית את הזמן לניסיון והוביל לריצות שבהן מאות ניסיונות נפרסו על פני כמעט שעותיים. זהו פתרון "רך" שמקשה על מתקפות בקנה מידה גדול תוך שמירה יחסית על חוויית משתמש (תלוי במדיניות).

Lockout יוצר עצירה כמעט מוחלטת לאחר מעט ניסיונות, ולכן יעיל מאוד נגד Brute-Force ממוקד; אך OWASP מזהיר ממנגנון נעילה שניתן לנצל ל-DoS על משתמשים (תוקף נועל חשבונות של אחרים).

מנגנונים אלו באים לידי ביטוי בטסטים 10 ו-13, וניתן לקחת כדוגמה את המשתמש weak_06. כאשר מוסיפים את מנגנון ה-Rate Limit, משך הזמן הדרוש על מנת להגיע לפריצה גדל משמעותית, וכאשר מוסיפים את מנגנון ה-Lockout, נמנעת פריצת משתמש זה לחלוטין (סיסתו מופיעה בקובץ הסיסמאות בשורה 512~ והמשתמש נחסם הרבה לפני).

כמו כן, צריך להבהיר שהצלחת מנגנון Rate Limit תלוי באופן ישיר לאופן התקיפה וכמות המשתמשים המותקפת. זאת אומרת, אילו נגדיר Rate Limit גבוהה מדי אל מול כמות המשתמשים המותקפת, מתקפות מסוג Password Spray יהיו מסוגלות "לעקוף" את מנגנון ההגנה, בכך שמספר הבקשות שנשלחות עבור כל משתמש בדקה יהיה קטן מהכמות הנדרשת בחסימה.

בנוסף, חשוב גם לקחת בחשבון את אופן ביצוע החסימה. בפרויקט שלנו, הוגדר במפורש לבצע חסימה לפי שם משתמש, אך בשרתי אמת לרוב מבצעים את החסימה לפי השילוב $\{IP: Username\}$ או אך ורק עבור ה-IP. באופן זה ניתן לחסום לגמרי מתקפות במידה והן מבוצעות על ידי אדם יחיד ולא על ידי שרתי התקפה.

CAPTCHA – חסם אוטומציה, אך לא פתרון קסם

בתכנון אבטחה תקין, CAPTCHA אמור להקשות על אוטומציה כאשר יש ריבוי ניסיונות כושלים. בפועל, בפרויקט שלנו, אפשרנו גישה למשתמש בעזרת נקודת קצה ייעודית. זה נעשה על מנת להמשיך את רצף ההתקפות.

עם זאת, חשוב להדגיש כי פריצה ל-CAPTCHA (וגם ל-TOTP) הינה מאוד קשה, וסיכויי פריצה הם כמעט ואפסיים.

Password Spraying – למה "Full Stacked" משנה את הכיסוי

לפי הגדרה, Password Spray שואף להימנע מנעילה ממוקדת על ידי פיזור ניסיונות.

ללא הגנות (Test23) התוקף ביצע $\sim 23k$ ניסיונות בשעתיים והצליח להשיג מספר הצלחות לאורך הזמן.

עם Fully Stacked (Test20), התוקף ביצע רק 128 ניסיונות לפני שקיבל סטטוס 429 / 423 / CAPTCHA אשר צמצמו את הכיסוי משמעותית.

כלומר, ההגנות לא רק "מעלות זמן", אלא מקטינות מרחב חיפוש אפקטיבי בתנאי זמן ריצה קבועים.

שימוש במשאבי CPU והשפעתו על Attempts/sec

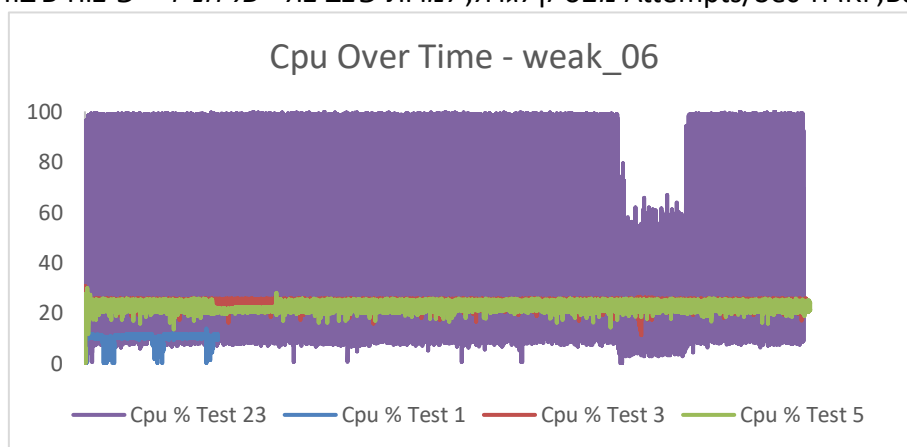
במערכת תקיפת Online, מספר הניסיונות לשנייה (attempts/sec) נקבע בפועל לפי צוואר הבקבוק;

נבחן כעת האם ההשהיה נשלטת על ידי עלות החישוב בצד השרת (CPU/זיכרון)? או שמא על ידי תקורה אחרת (רשת, HTTP, תורים, נעילות, IO), או על ידי מנגנוני ההגנה (Rate Limiting/Lockout/CAPTCHA)?

כאשר משתמשים ב-Hash מהיר כמו SHA256, עלות החישוב לכל ניסיון קטנה מאוד, ולכן השרת מסוגל לבצע ניסיונות רבים לשנייה עד שהוא נתקל במגבלה אחרת – קצב בקשות HTTP, ניהול חיבורים, תזמון Threads, או פשוט מגבלת הלקוח / מספר חיבורים במקביל (פחות בא לידי ביטוי בניסוי שלנו).

דבר זה מתיישב עם תוצאות הניסוי שלנו. ניתן להבחין כי ללא הגנות (Test01) התקיפה מגיעה לעשרות ניסיונות לשנייה בריצה רציפה, כלומר לא נראה שהשרת "נחנק" מהחישוב עצמו אל שמגבלות שכבת ה-HTTP קובעות את התקרה.

- כאשר משתמשים ב-BCrypt או ב-Argon2ID, העלות החישובית לכל ניסיון גבוהה בהרבה, ולכן ה-CPU בצד השרת הופך לצוואר בקבוק מרכזי. כאן, ה-Attempt/Sec יורד משמעותית, כי כל ניסיון "תופס" זמן CPU ארוך יותר, והשרת יכול לבצע פחות אימותים במקביל לפני רוויה. בתוצאות שלנו, דבר זה ניכר מאוד. BCRYPT ו-Argon2ID מורידים את הקצב לרמות של ניסיונות בודדים בשנייה בריצות רציפות.
- חשוב לקחת בחשבון את הממד הנוסף ש-Argon2ID מוסיף – לא רק CPU אלא גם זיכרון ורוחב פס זכרון. לכן גם אם מוסיפים יותר threads, השיפור אינו תמיד לינארי; עלולים להגיע לרוויה של Memory Bandwidth, ואז ה-Attempts/Sec מפסיק לגדול, למרות שבביכול "על הנייר" יש כוח עיבוד זמין.



השפעת קובץ הסיסמאות

בניסויים ראינו שהצלחות בתקיפות Brute-Force ו-Password Spraying התקבלו כמעט רק עבור חשבונות בעלי סיסמאות חלשות, בעוד שחשבונות בינוניים וחזקים לא נפרצו במסגרת מגבלות הניסוי (בעיקר מגבלת זמן ריצה).

חשוב להדגיש, שמסקנה זו תלוי במידה רבה באיכות ובסדר של קובץ הסיסמאות שבו השתמשנו. אם ה-Dictionary אינו מכיל ייצוג מספק של סיסמאות בינוניות / חזקות נפוצות, או אם הסדר שלו מציב אותן מאוחר מאוד, אז גם תוקף אמיתי עשוי לאבד את חלון הזמן על ניסיונות בעלי סיגוי נמוך לפני שמגיע לאיזורים רלוונטים יותר. לכן אי הצלחה בתקיפת medium / strong תחת מגבלת זמן אינה בהכרח מעידה שהן בלתי ניתנות לניחוש.

כמו כן, בעת הצבת נהלי הרשמה וסוג סיסמאות (לדוגמה: הגבלת משתמשים לסיסמאות חזקות מאוד בלבד בעת הרשמה), צריך לקחת בחשבון כי סיסמאות אלו ככל הנראה אינן נפוצות, ולכן תקיפות עלולות להמשך זמן רב. אי לכך, צריך להתאים את מצב ההגנות ולוודא שהשרת יצליח לעמוד בדרישות עומס המערכת הן מבחינה חישובית והן מבחינת עומס בקשות, על מנת שחווית משתמשים לא תפגע.

תוקף הניסוי – מגבלות ומקורות טעות

- עוצמת חומרה ועומסים משתנים – Latency ו-Throughput תלויים בעומס המערכת / שרת. הדבר בולט במיוחד אצלינו בהשוואת Pepper vs No Pepper. כמו כן, ייתכן ובאשר ישנה חומרה חזקה יותר, מספר הבקשות לשנייה יעלה כאשר אין הגנות.
- גודל מדגם המשתמשים – בניתוח זה, השתמשנו בקובץ הסיסמאות RockYou¹¹, ובעקבות ברירות המחדל של הפרויקט, רוב הניתוח (וגם ההצלחות) היו על קבוצת המשתמשים בעלי הסיסמאות החלשות. הרחבת הגבלת ההודעות או משך הזמן, עלולה הייתה להביא לנו תוצאות טיפה יותר מעניינות גם עבור משתמשים בעלי סיסמאות בדרגה Medium / Strong.
- כלי תקיפה פשוטים – המימוש לא בהכרח מייצג תוקף אופטימלי (למשל תזמון חכם מול Rate Limit, סנכון לשעון ב-TOTP, פתרון CAPTCHA וכו').
- הגבלת זמן פשוטה – החסימה Rate Limit בוצעה לפי Username (ספציפית אצלינו בעזרת מחרוזת `{IP: Username}`, אך ניתן גם לייעל וליצור מגבלה לפי IP, אשר הייתה מונעת את המשך הבקשות במתקפות Password Spray.

שיקולים אתיים

הפרויקט בוצע בסביבה ניסויית מבוקרת ועל חשבונות סינתטיים, ללא פגיעה במשתמשים אמיתיים וללא ניסיון גישה למערכות צד ג'. עקרונות האתיקה הותאמו למסגרות מקובלות במחקר אבטחת מידע (ACM Code Of Ethics¹²)

יישום מעשי בפרויקט:

- בוצעו ניסיונות רק מול שירות ניסויי בבעלות / שליטה מלאה של הצוות.
- הוגדרו מגבלות עצירה (שעתיים / $50k$ ניסיונות) כדי למנוע עומס לא נשלט.
- תועדו תצורות ולוגים לצורך שקיפות ושחזור, ללא איסוף מידע אישי ממשתמשים אמיתיים.

¹¹RockYou.txt - קובץ המכיל סיסמאות נפוצות.

¹²ACM – Code of Ethics and Professional Conduct (2018)

מסקנות ועבודה עתידית

מסקנות

1. ללא הגנות סביב האימות, בחירת אלגוריתם גיבוב הוא גורם מכריע בקצב התקיפה. SHA256 מאפשר קצב ניסיונות גבוה משמעותית, בעוד BCrypt / Argon2ID מורידים את הקצב משמעותית.
2. Rate Limiting הוא מנגנון מרכזי שמגדיל את ה-Time To Success, באופן דרמטי, גם כאשר ישנו Hash מהיר.
3. Lockout יעיל מאוד נגד התקפות Brute-Force ממוקדות, אך דורש טיפול זהיר בגלל סיכון ל-DoS על משתמשים.
4. TOTP חסם אוטומציה בצורה מוצלחת בניסוי, אך יש להציגו הגנה משלימה ולא יחידה.
5. Password Spraying, הגנה "Fully Stacked" משנה בעיקר את הכיסוי האפשרי (כמה ניסיונות אפשר לבצע), ולא רק את ה-latency של ה-hash. אי לכך, מאוד יעיל מול מודל תקיפה זו.
6. בחירה ב Argon2Id / BCrypt יוצרת עלות CPU שמקטינה את ה-Attempts / Sec באופן טבעי, ולכן משפרת עמידות מול התקפות Brute-Force גם בלי הגנות נוספות.

עבודה עתידית

1. הרחבת שטח הניסוי – הגדלת מספר הניסיונות הדיפולטיבי עבור כל תקיפה כך שניתן יהיה לבדוק גם סיסמאות ברמת Medium / Strong.
2. בדיקת מדיניות Rate Limiting מתקדמת, למשל שילוב של Per IP + Per Account.

1. OWASP Password Storage Cheat Sheet
https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html
2. RFC 9106 - Argon2 Memory-Hard Function for Password Hashing and Proof-of-Work Applications
<https://datatracker.ietf.org/doc/rfc9106/>
3. Rainbow Tables
<https://www.beyondidentity.com/glossary/rainbow-table-attack>
4. Authentication Cheat Sheet
https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html
5. Blocking Brute Force Attacks
https://owasp.org/www-community/controls/Blocking_Brute_Force_Attacks
6. RFC6238 - TOTP: Time-Based One-Time Password Algorithm
<https://datatracker.ietf.org/doc/html/rfc6238>
7. Peppering – Password Storage Cheat Sheet
https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html?#peppering
8. ACM – Code of Ethics and Professional Conduct (2018)
<https://www.acm.org/binaries/content/assets/about/acm-code-of-ethics-and-professional-conduct.pdf>